

# FedEx AI Logistics Command Center

generate\_ai\_insights.py — Hugging Face LLM script (matched to FedEx\_project.ipynb dataset)

---

## What this script does

Loads the cleaned FedEx logistics dataset (MySQL or CSV) using the same cleaning steps as your notebook, calculates real KPIs in Python (on-time %, delay days, freight cost, shipment mode mix, top country/vendor), sends only the KPI summary to a pretrained Hugging Face model (Qwen/Qwen2.5-3B-Instruct), robustly parses the model's JSON insight, and saves `ai_insights.csv` for Power BI to read.

Install once: `pip install pandas numpy transformers torch accelerate sentencepiece  
sqlalchemy mysql-connector-python`

Run with: `python generate_ai_insights.py`

## Full Script

```
"""
FedEx AI Logistics Command Center
-----
Matched to your actual FedEx_project.ipynb dataset (SCMS Delivery History /
logistics_dataset table). Reads the cleaned data -> calculates real KPIs in
Python -> sends KPI summary to a pretrained Hugging Face LLM -> parses
structured JSON insight -> saves ai_insights.csv for Power BI.

INSTALL (run once in terminal):
    pip install pandas numpy transformers torch accelerate sentencepiece sqlalchemy
    mysql-connector-python

USAGE:
    python generate_ai_insights.py
"""

import os
import re
import json
import pandas as pd
import numpy as np
from transformers import pipeline

# =====
# 1. CONFIG - EDIT FOR YOUR MACHINE
# =====
OUTPUT_PATH = r"D:\FedEx_AI_Project\ai_insights.csv" # Power BI reads this file
MODEL_ID = "Qwen/Qwen2.5-3B-Instruct" # small, free, runs locally

# Set to True to pull straight from your MySQL DB (same as your notebook).
# Set to False to read from a CSV export instead.
USE_MYSQL = True
CSV_PATH = r"D:\FedEx_AI_Project\dataset.csv"

# =====
# 2. LOAD DATA (same source as your notebook)
# =====
if USE_MYSQL:
    from sqlalchemy import create_engine
    engine = create_engine("mysql+mysqlconnector://root:shruti@localhost/fedex")
    df = pd.read_sql("SELECT * FROM logistics_dataset", engine)
else:
    df = pd.read_csv(CSV_PATH)

print(f"Loaded {len(df)} rows.")

# =====
# 3. CLEAN DATA (mirrors the cleaning steps in your notebook)
# =====
ex_df = df.copy()

# Fill missing Shipment Mode with the mode (as in your notebook)
if "Shipment Mode" in ex_df.columns:
    shipment_mode_fill = ex_df["Shipment Mode"].mode()[0]
    ex_df["Shipment Mode"] = ex_df["Shipment Mode"].fillna(shipment_mode_fill)

# Fill missing insurance with 0 (as in your notebook)
if "Line Item Insurance (USD)" in ex_df.columns:
    ex_df["Line Item Insurance (USD)"] = ex_df["Line Item Insurance (USD)"].fillna(0)
ex_df.drop_duplicates(inplace=True)

# Convert date columns
date_cols = [
    "PQ First Sent to Client Date", "PO Sent to Vendor Date",
    "Scheduled Delivery Date", "Delivered to Client Date", "Delivery Recorded Date",
]
for col in date_cols:
```

```

    if col in ex_df.columns:
        ex_df[col] = pd.to_datetime(ex_df[col], errors="coerce")

# Convert numeric columns
numeric_cols = ["Line Item Quantity", "Line Item Value", "Weight (Kilograms)",
                "Freight Cost (USD)", "Line Item Insurance (USD)"]
for col in numeric_cols:
    if col in ex_df.columns:
        ex_df[col] = pd.to_numeric(ex_df[col], errors="coerce")

# Cap Line Item Value outliers using IQR (same as your notebook)
if "Line Item Value" in ex_df.columns:
    Q1 = ex_df["Line Item Value"].quantile(0.25)
    Q3 = ex_df["Line Item Value"].quantile(0.75)
    IQR = Q3 - Q1
    lower, upper = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
    ex_df["Line Item Value"] = ex_df["Line Item Value"].clip(lower, upper)

clean_ex_df = ex_df.copy()

# =====
# 4. PYTHON CALCULATES THE KPIS
# NOTE: this dataset has no Revenue/Profit/Delivery-Status columns.
# On-time delivery and delay are derived from the two date columns.
# Line Item Value is used as an order-value proxy (not "revenue").
# =====
total_shipments = len(clean_ex_df)

order_value_total = round(clean_ex_df["Line Item Value"].sum(), 2)
freight_cost_total = round(clean_ex_df["Freight Cost (USD)"].sum(skipna=True), 2)
freight_cost_avg = round(clean_ex_df["Freight Cost (USD)"].mean(skipna=True), 2)
avg_weight = round(clean_ex_df["Weight (Kilograms)"].mean(skipna=True), 2)

# Delay in days = Delivered date - Scheduled date (positive = late, negative/0 = on time or early)
clean_ex_df["Delay Days"] = (
    clean_ex_df["Delivered to Client Date"] - clean_ex_df["Scheduled Delivery Date"]
).dt.days

valid_delay = clean_ex_df["Delay Days"].dropna()
delayed_shipments = int((valid_delay > 0).sum())
on_time_shipments = int((valid_delay <= 0).sum())
on_time_percentage = round((on_time_shipments / len(valid_delay)) * 100, 2) if len(valid_delay)
    else 0
average_delay_days = round(valid_delay[valid_delay > 0].mean(), 2) if delayed_shipments else 0

# Shipment mode mix
mode_pct = (clean_ex_df["Shipment Mode"].value_counts(normalize=True) * 100).round(2)
air_percentage = float(mode_pct.get("Air", 0))
truck_percentage = float(mode_pct.get("Truck", 0))

# Top country / vendor by shipment count
top_country = clean_ex_df["Country"].value_counts().idxmax() if "Country" in clean_ex_df.columns
    else "N/A"
top_vendor = clean_ex_df["Vendor"].value_counts().idxmax() if "Vendor" in clean_ex_df.columns else
    "N/A"

# Health score (no profit column available, so this is based on on-time %,
# delay severity, and how reliant operations are on the costlier Air mode)
health_score = 100
if on_time_percentage < 95:
    health_score -= 10
if on_time_percentage < 90:
    health_score -= 10
if on_time_percentage < 80:
    health_score -= 10
if average_delay_days > 15:
    health_score -= 10
if air_percentage > 60:
    health_score -= 10
health_score = max(health_score, 0)

```

```

kpi_summary = f"""
Total Shipments: {total_shipments}
Total Order Value (Line Item Value): {order_value_total}
Total Freight Cost (USD): {freight_cost_total}
Average Freight Cost (USD): {freight_cost_avg}
Average Weight (KG): {avg_weight}
On-Time Delivery: {on_time_percentage}%
Delayed Shipments: {delayed_shipments} out of {len(valid_delay)}
Average Delay (for delayed shipments): {average_delay_days} days
Air Shipments: {air_percentage}%
Truck Shipments: {truck_percentage}%
Top Country by Volume: {top_country}
Top Vendor by Volume: {top_vendor}
Health Score: {health_score}
"""

print("KPI Summary:")
print(kpi_summary)

# =====
# 5. LOAD PRETRAINED HUGGING FACE LLM (no training needed)
# =====
print(f>Loading model: {MODEL_ID} ...")
llm = pipeline(
    task="text-generation",
    model=MODEL_ID,
    device_map="auto",
)
print("LLM loaded successfully.")

# =====
# 6. BUILD THE PROMPT (chat format works much better than raw text
#     for instruction models like Qwen2.5-Instruct)
# =====
system_msg = (
    "You are a Senior Logistics Director. You always reply with STRICT, VALID "
    "JSON only. No markdown, no code fences, no explanation outside the JSON."
)

user_msg = f"""Analyze these FedEx logistics KPIs:
{kpi_summary}
Return ONLY this JSON object, filled in, with no extra text:
{{
    "health_score": {health_score},
    "status": "",
    "top_risk": "",
    "opportunity": "",
    "recommendation": "",
    "executive_summary": ""
}}
"""

messages = [
    {"role": "system", "content": system_msg},
    {"role": "user", "content": user_msg},
]

response = llm(messages, max_new_tokens=400, do_sample=False)

raw_output = response[0]["generated_text"]
if isinstance(raw_output, list):
    ai_text = raw_output[-1]["content"]
else:
    ai_text = str(raw_output)

print("Raw model output:")
print(ai_text)

# =====
# 7. ROBUST JSON EXTRACTION
# =====

```

```

def extract_json(text):
    match = re.search(r"\{.*\}", text, re.DOTALL)
    if not match:
        return None
    try:
        return json.loads(match.group(0))
    except json.JSONDecodeError:
        return None

parsed = extract_json(ai_text)

if parsed is None:
    print("WARNING: Could not parse JSON from model output. Using fallback values.")
    parsed = {
        "health_score": health_score,
        "status": "Unknown",
        "top_risk": "Unable to generate insight (parsing failed).",
        "opportunity": "N/A",
        "recommendation": "Re-run analysis or check model output.",
        "executive_summary": ai_text.strip()[:500],
    }

# =====
# 8. BUILD FINAL DATAFRAME FOR POWER BI
# =====
result = pd.DataFrame([
    {
        "HealthScore": parsed.get("health_score", health_score),
        "Status": parsed.get("status", ""),
        "TotalOrderValue": order_value_total,
        "TotalFreightCost": freight_cost_total,
        "OnTimePercentage": on_time_percentage,
        "AverageDelayDays": average_delay_days,
        "TopCountry": top_country,
        "TopVendor": top_vendor,
        "Risk": parsed.get("top_risk", ""),
        "Opportunity": parsed.get("opportunity", ""),
        "Recommendation": parsed.get("recommendation", ""),
        "ExecutiveSummary": parsed.get("executive_summary", "")
    }
])

os.makedirs(os.path.dirname(OUTPUT_PATH), exist_ok=True)
result.to_csv(OUTPUT_PATH, index=False)

print(f"ai_insights.csv updated successfully at: {OUTPUT_PATH}")
print(result.to_string(index=False))

```